MCA -I Sem – I

Subject

Python Programming Lab[IT11L]

For Academic Year 2025-2026

Submitted By:
Rushikesh Ganesh Yadav

Department of Master of Computer Application

CERTIFICATE

This is to certify that, Mr./Ms. **Rushikesh Ganesh Yadav** of MCA (SEM-I) Class, Roll No. **MC2527261**, Exam Seat No. , has completed all the Practical work of the subject "IT11L: Python Practical Assignments" satisfactorily in the MCA programme as prescribed by Savitribai Phule Pune University in the academic year 2025- 2026.

Date :

Course Coordinator **HOD, MCA** **Director**

Examiner 1 Sign **Examiner 2 Sign**

INDEX

Subject: - [IT 11L] Python Lab

ASSIGNMENT 1

Sr. No.	Practical Name	Date	Sign
1.	Write a python program to print the following string in a specific format? Out put : Twinkle, twinkle, little star, How I wonder what you are! Up above the world so high, Like a diamond in the sky. Twinkle, twinkle, little star, How I wonder what you are		
2	Write a program to perform arithmetic operations.		
3	Write a program using list to perform below operation: a. Slicing b. Concatenation c. Repetition		
4	Write a program using tuple to perform below operation: a. Slicing b. Concatenation c. Repetition		
5	Write a program using dictionary to access value using keys .		
6.	Write a program using set: a. Create empty set b. Assign value to set c. Adding element d. Removing element		
7.	. Write a program to get the largest number from the given list ?		

8.	Write a program to find out Factorial of a given Number		
9.	Write a program to find given number is prime or not		
10.	Write a program to identify total number of digits present in given number		

11.	Write a program to display Fibonacci series up to a given number by using range function		
12.	Write a program using for loop with: String, Integer, Enumerate, Nested, For List, Dictionary, Tuple, Zip()		
13.	Write a program to perform Loop manipulation using pass, continue, break and else		
14.	Write a program to perform Comprehensions on List Dictionaries.		
15.	Write a program to Find all the numbers from 1-500 that have three in them.		
16..	Write a python program for numbers=range(20) produce a list containing the word "even" if the number is even or the word "odd" if the number is odd		
17	Find the common numbers in two list 1[1,3,4,5,6,7] 2[5,2,8,7,1,3]		

ASSIGNMENT 2

1	Implement a nested function that calculates compound interest using non-local variables		
2	Write a program using lambda to filter even numbers from a list		
3.	Write a program using lambda to find area of triangle		
4.	Create a generator function to yield Fibonacci numbers up to a certain limit		
5.	Develop a custom Python module with functions for basic statistical operations (mean, median, mode)		
6.	rite a program to handle file reading errors and log them to a separate file		
7	reate a custom exception class AgeException and use it to validate user input		

ASSIGNMENT 3

1	Write a simple Python class named Student and display its type. Also display the dict attribute keys and values of the module attribute of the Student class.		
2	Write a Python class Student with attributes, student name and marks. Modify any attribute value of the class and show the original and modified values o attributes.		
3	Write a Python class to convert an integer to a Roman numeral.		
4	Write a Python class to convert a Roman numeral to an integer		
5	W ite a program to create a class Shape with methods called get_perimeter and get area. Create a subclass called Circle that overrides the get_perimeter and get area methods o calculate the area and erimeter of a circle.		
6	rite a program to create a Vehicle class hierarchy. The base class should be ehicle, with subclasses Truck, Car, and Motorcycle. Each subclass should ave properties such as make, model, year, and fuel_type. Implement methods for calculatin fuel efficienc , distance traveled, and maximum s eed.		
7	Implement a class BankAccount with methods for deposit, withdrawal, and dis la in the account balance.		
8	Develop a hierarchy of shapes (e.g., Shape, Circle, Rectangle) with area and perimeter methods.		
9	Overload the + operator for concatenating two custom string objects		

10	Create a static method to validate user credentials (e.g., email format, password len h		
11	Implement a class Library containing a list of Book objects		
12	Demonstrate method overriding in a superclass-subclass relationship for a payment		
13	Write a program to validate a password that meets certain criteria (length, uppercase, dioit s ecial character .		
14	Parse email addresses from a given text and classify them by domain		
15	Create threads to perform independent tasks such as downloading multiple files Simultaneousl		
16	Write a Python program to create two threads. The first thread will print even numbers, and the second thread will print odd numbers. Demonstrate the conce t of multithreadin		
17	Use multithreading to compute factorials of multiple numbers and log the results into a file.		

ASSIGNMENT 4

1.	Develop a Python program to create, read, update, and delete documents in MongoDB a collection.		
2.	Write a program to insert valid data into MongoDB using exception handling for errors.		
3.	Use MongoDB's aggregation pipeline to find the average salary of employees in a collection.		
4.	Develop a Python program to create a database name employee. With the collection name empinfo. Add 5 douments in the collection. Perform read, update, and delete operations on documents in a MongoDB.		

ASSIGNMENT 5

1	Create a Django project with models for Product and Category, along with views to display them on a webpage.		
2.	mplement user login, logout, and registration functionality in a Django application.		
3.	Build a REST API for a task management system where users can create, update, and delete tasks		

Roll No: MC2527261
Student Name: Rushikesh Ganesh Yadav
Subject Code: IT11L
Subject Name: Python Practical Lab Assignments

Assignment-1

1. Write a python program to print the following string in a specific format?

Out put :

Twinkle, twin

How I wonder what you are!

Up above the world so high,

Like a diamond in the sky.

Twinkle, twinkle, little star,

How I wonder what you are

Ans-

```
print("Twinkle, twin")
print("How I wonder what you are!")
print("Up above the world so high,")
print("Like a diamond in the sky.")
print("Twinkle, twinkle, little star,")
print("How I wonder what you are")
```

2. Write a program to perform arithmetic operations.

Ans-

```
# Taking input
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
# Performing operations
print("Addition (+):", a + b)
print("Subtraction (-):", a - b)
print("Multiplication (*):", a * b)
print("Division (/):", a / b)
```

Output-

```
Enter first number: 10
Enter second number: 4
Addition (+): 14
Subtraction (-): 6
Multiplication (*): 40
Division (/): 2.5
```

3.Write a program using list to perform below operation:

a. Slicing

b. Concatenation

c. Repetition

Ans-

```
# Creating lists
list1 = [10, 20, 30, 40, 50, 60]
list2 = ['A', 'B', 'C']
# a. Slicing
print("Slicing list1[1:4]:", list1[1:4])
# b. Concatenation
print("Concatenation list1 + list2:", list1 + list2)
# c. Repetition
print("Repetition list2 * 3:", list2 * 3)
```

Ouput-

```
Slicing list1[1:4]: [20, 30, 40]
Concatenation list1 + list2: [10, 20, 30, 40, 50, 60, 'A', 'B', 'C']
Repetition list2 * 3: ['A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C']
```

4.Write a program using tuple to perform below operation:

d. Slicing

e. Concatenation

f. Repetition

Ans-

```
# Tuple operations: Slicing, Concatenation, Repetition
# Creating tuples
t1 = (10, 20, 30, 40, 50, 60)
t2 = ('A', 'B', 'C')
# a. Slicing
print("Slicing t1[1:4]:", t1[1:4])
# b. Concatenation
print("Concatenation t1 + t2:", t1 + t2)
# c. Repetition
print("Repetition t2 * 3:", t2 * 3)
```

Output-

```
Slicing t1[1:4]: (20, 30, 40)
Concatenation t1 + t2: (10, 20, 30, 40, 50, 60, 'A', 'B', 'C')
Repetition t2 * 3: ('A', 'B', 'C', 'A', 'B', 'C', 'A', 'B', 'C')
```

5.Write a program using dictionary to access value using keys .

Ans-

```
# Creating a dictionary
student = {
    "name": "Avantika",
    "age": 20,
    "course": "Computer Science",
    "marks": 85
}
# Accessing values using keys
print("Name:", student["name"])
print("Age:", student["age"])
print("Course:", student["course"])
print("Marks:", student["marks"])
```

Output-

```
Name: Avantika
Age: 20
Course: Computer Science
Marks: 85
```

6. Write a program using set:

a. Create empty set

b. Assign value to set

c. Adding element

d. Removing element

Ans-

```
# a. Create empty set
my_set = set()
print("Empty set:", my_set)
# b. Assign values to set
my_set = {10, 20, 30}
print("Set after assigning values:", my_set)
# c. Adding an element
my_set.add(40)
print("Set after adding element 40:", my_set)
# d. Removing an element
my_set.remove(20)
print("Set after removing element 20:", my_set)
```

Output-

```
Empty set: set()
Set after assigning values: {10, 20, 30}
Set after adding element 40: {40, 10, 20, 30}
Set after removing element 20: {40, 10, 30}
```

7. Write a program to get the largest number from the given list ?

Ans-

```
# Program to find the largest number in a list

numbers = [10, 55, 23, 87, 42, 19]

largest = max(numbers)

print("The largest number in the list is:", largest)
```

Output:-

The largest number in the list is: 87

8. Write a program to find out Factorial of a given Number

Ans-

```
# Program to find factorial of a number using loop
num = int(input("Enter a number: "))
fact = 1

for i in range(1, num + 1):
    fact *= i

print("Factorial of", num, "is", fact)
```

Output:

```
Enter a number: 5
Factorial of 5 is 120
```

9. Write a program to find given number is prime or not.

Ans-

```
num = int(input("Enter a number: "))
if num <= 1:
    print(num, "is not a prime number")
else:
    for i in range(2, num):
        if num % i == 0:
            print(num, "is not a prime number")
            break
    else:
        print(num, "is a prime number")
```

Output-

```
Enter a number: 5
5 is a prime number
```

10. **Write a program to identify total number of digits present in given number**.

Ans-

```
num = int(input("Enter a number: "))

count = 0

while num != 0:
    num = num // 10
    count += 1

print("Total number of digits:", count)
```

Output-

```
Enter a number: 1234
Total number of digits: 4
```

11. Write a program to display Fibonacci series up to a given number by using range function.

Ans-

```
num = int(input("Enter the number of terms: "))

a, b = 0, 1

print("Fibonacci Series:")

for i in range(num):
    print(a, end=" ")
    a, b = b, a + b
```

Output:-

```
Enter the number of terms: 6
Fibonacci Series:
0 1 1 2 3 5
```

12. Write a program using for loop with:String,Integer,Enumerate,Nested For,List,Dictionary,Tuple,Zip()

Ans-

```
# 1. For Loop with String
text = "HELLO"
print("For loop with String:")
for ch in text:
    print(ch, end=" ")
 print("\n")

# 2. For Loop with Integer (range)
print("For loop with Integer (range):")
for i in range(1, 6):
    print(i, end=" ")
print("\n")

# 3. For Loop with enumerate()
fruits = ["apple", "banana", "mango"]
print("For loop with enumerate():")
for index, item in enumerate(fruits):
    print(index, item)
print()

# 4. Nested For Loop
print("Nested For Loop:")
for i in range(1, 3):
    for j in range(1, 4):
        print(f"i={i}, j={j}")
print()

# 5. For Loop with List
numbers = [10, 20, 30, 40]
print("For loop with List:")
for n in numbers:
    print(n, end=" ")
print("\n")

# 6. For Loop with Dictionary
student = {"name": "Avantika", "age": 20, "marks": 85}
print("For loop with Dictionary:")
for key in student:
    print(key, ":", student[key])
print()

# 7. For Loop with Tuple
t = (5, 10, 15, 20)
```

```
print("For loop with Tuple:")
for val in t:
    print(val, end=" ")
print("\n")

# 8. For Loop with zip()
names = ["Avi", "Rahul", "Siya"]
marks = [90, 85, 88]
print("For loop with zip():")
for n, m in zip(names, marks):
    print(n, "=", m)
```

Output-

```
For loop with String:
H E L L O

For loop with Integer (range):
1 2 3 4 5

For loop with enumerate():
0 apple
1 banana
2 mango

Nested For Loop:
i=1, j=1
i=1, j=2
i=1, j=3
i=2, j=1
i=2, j=2
i=2, j=3

For loop with List:
10 20 30 40

For loop with Dictionary:
name : Avantika
age : 20
marks : 85

For loop with Tuple:
5 10 15 20

For loop with zip():
Avi = 90
Rahul = 85
Siya = 88
```

13. Write a program to perform Loop manipulation using pass, continue, break and else

Ans-

```
# Sample list
numbers = [1, 2, 3, 4, 5, 6]

print("Using pass:")
for num in numbers:
    if num == 3:
     pass  # Does nothing
print(num, end=" ")
print("\n")

print("Using continue:")
for num in numbers:
    if num == 3:
        continue  # Skip the rest of the loop for num=3
    print(num, end=" ")
print("\n")

print("Using break:")
for num in numbers:
    if num == 4:
        break  # Exit loop when num=4
    print(num, end=" ")
print("\n")

print("Using else with for loop:")
for num in numbers:
    if num > 10:
        break
    print(num, end=" ")
else:
    print("\nLoop completed without break")
```

Output-

```
Using pass:
1 2 3 4 5 6

Using continue:
1 2 4 5 6

Using break:
1 2 3
```

Using else with for loop:
1 2 3 4 5 6
Loop completed without break

14. Write a program to perform Comprehensions on List Dictionaries.

Ans-

```
# 1. List Comprehension
# Example: Create a list of squares of numbers from 1 to 10
squares = [x**2 for x in range(1, 11)]
print("List of squares:", squares)

 # Example: List of even numbers from 1 to 10
even_numbers = [x for x in range(1, 11) if x % 2 == 0]
print("List of even numbers:", even_numbers)

# 2. Dictionary Comprehension
# Example: Create a dictionary where key = number, value = square of number
square_dict = {x: x**2 for x in range(1, 6)}
print("Dictionary of squares:", square_dict)

# Example: Dictionary of numbers and their cubes (only for even numbers)
cube_dict = {x: x**3 for x in range(1, 6) if x % 2 == 0}
print("Dictionary of cubes for even numbers:", cube_dict)
```

Output-

```
List of squares: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
List of even numbers: [2, 4, 6, 8, 10]
Dictionary of squares: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
Dictionary of cubes for even numbers: {2: 8, 4: 64}
```

15. Write a program to Find all the numbers from 1-500 that have three in them.

Ans-

```
# Empty list to store numbers containing 3
numbers_with_3 = []

# Loop from 1 to 500
for num in range(1, 501):
    if '3' in str(num):  # Check if digit 3 is in the number
        numbers_with_3.append(num)

# Display the result
print("Numbers from 1 to 500 containing the digit 3:")
print(numbers_with_3)
```

Output-

Numbers from 1 to 500 containing the digit 3:
[3, 13, 23, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 43, 53, 63, 73, 83, 93, 103, 113, 123, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 143, 153, 163, 173, 183, 193, 203, 213, 223, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 243, 253, 263, 273, 283, 293, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 403, 413, 423, 430, 431, 432, 433, 434, 435, 436, 437, 438, 439, 443, 453, 463, 473, 483, 493]

16. Write a python program for numbers=range(20) produce a list containing the word "even" if the number is even or the word "odd" if the number is odd

Ans-

```
# Using list comprehension
numbers = range(20)

even_odd_list = ["even" if num % 2 == 0 else "odd" for num in numbers]

print(even_odd_list)
```

Output-

```
['even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd',
 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd']
```

17. Find the common numbers in two list 1[1,3,4,5,6,7] 2[5,2,8,7,1,3]

Ans-

```
# Given lists
list1 = [1, 3, 4, 5, 6, 7]
list2 = [5, 2, 8, 7, 1, 3]

# Using list comprehension to find common elements
common_numbers = [num for num in list1 if num in list2]

print("Common numbers in the two lists:", common_numbers)
```

Output-

Common numbers in the two lists: [1, 3, 5, 7]

UNIT-II

Python Practical Assignment-2

Que.1. Implement a nested function that calculates compound interest using non-local variables.

Ans-

```
def compound_interest_calculator(principal, rate, time):
    interest = 0  # This will be modified by the nested function

    def calculate():
        nonlocal interest  # Allows modifying variable from outer function
        amount = principal * (1 + rate/100) ** time
        interest = amount - principal
        return amount, interest

    amount, interest = calculate()
    return amount, interest
# Example usage
total_amount, total_interest = compound_interest_calculator(10000, 5, 2)
print("Total Amount:", total_amount)
print("Compound Interest:", total_interest)
```

Output-

Total Amount: 11025.0
Compound Interest: 1025.0

Que.2. Write a program using lambda to filter even numbers from a list.
Ans-

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print("Even numbers:", even_numbers)
```

Output-
Even numbers: [2, 4, 6, 8, 10]

Que.3. Write a program using lambda to find area of triangle.

Ans-

```
# Lambda function to calculate area of a triangle
area_triangle = lambda base, height: 0.5 * base * height
# Example
base = 10
height = 5
area = area_triangle(base, height)
print("Area of Triangle:", area)
```

Output-

Area of Triangle: 25.0

Que.4. Create a generator function to yield fibonacci numbers up to a certain limit.

Ans-

```
def fibonacci(limit):
    a, b = 0, 1
    while a <= limit:
        yield a
        a, b = b, a + b

# Example usage
for num in fibonacci(50):
    print(num)
```

Output-

0
1
1
2
3
5
8
13
21
34

Que.5. Develop a cutom python module with functions for basic statistical operations(mean,median,mode)

Ans-

```
# mystats.py

def mean(numbers):
    return sum(numbers) / len(numbers)

def median(numbers):
    numbers = sorted(numbers)
    n = len(numbers)
    mid = n // 2

    if n % 2 == 0:
        return (numbers[mid - 1] + numbers[mid]) / 2
    else:
        return numbers[mid]

def mode(numbers):
    freq = {}

    for num in numbers:
        freq[num] = freq.get(num, 0) + 1

    max_count = max(freq.values())
    modes = [k for k, v in freq.items() if v == max_count]

    if len(modes) == 1:
        return modes[0]
    return modes  # returns list if more than one mode
```

Create another Python file (example: test_stats.py) and import your module:

```
import mystats

data = [2, 4, 4, 6, 8, 4, 10]

print("Mean:", mystats.mean(data))
print("Median:", mystats.median(data))
print("Mode:", mystats.mode(data))
```

Output-

Mean: 5.428571428571429
Median: 4
Mode: 4

Que.6. Write a program to handle file reading errors and log them to a seperate file.
Ans-

```
def read_file(filename):
    try:
        with open(filename, "r") as file:
            data = file.read()
            print("File content:\n", data)

    except FileNotFoundError as e:
        log_error(str(e))
        print("Error: File not found!")

    except PermissionError as e:
        log_error(str(e))
        print("Error: Permission denied!")

    except Exception as e:
        log_error(str(e))
        print("An unexpected error occurred.")
def log_error(message):
    with open("error_log.txt", "a") as log:
        log.write(message + "\n")


# Test the function
read_file("sample.txt")
```

Output-
Error: File not found!
[Errno 2] No such file or directory: 'sample.txt'

Que.7. Create a custom exception class AgeException and use it to validate user input.

Ans-

```
# Creating custom exception
class AgeException(Exception):
    def __init__(self, message):
        super().__init__(message)


def check_age():
    try:
        age = int(input("Enter your age: "))

        if age < 0 or age > 120:
            raise AgeException("Invalid age! Age must be between 0 and 120.")

        print("Valid age entered:", age)

    except AgeException as e:
        print("Age Error:", e)

    except ValueError:
        print("Please enter a valid number for age.")


# Run the function
check_age()
```

Output-

Enter your age: 25
Valid age entered: 25s

UNIT-III

Python Practical Assignment-3

Question 1 Write a simple Python class named Student and display its type. Also display the dict attribute keys and values of the module attribute of the Student class.

Program:
```
class Student:
    pass

a = Student() print(type(a)) print(Student.__dict__.keys()) print(Student.__module__)
```

Output:
```
<class '__main__.Student'>
dict_keys(['__module__','__dict__','__weakref__','__doc__'])
__main__
```

Question 2 Write a Python class Student with attributes, student name and marks. Modify any attribute value of the class and show the original and modified values o attributes.

Program:
```
class Student:     def __init__(self,name,marks):
        self.name=name          self.marks=marks

s=Student("Rahul",85) print("Original:",s.name,s.marks)
s.name="Amit"
s.marks=92 print("Modified:",s.name,s.marks)
```

Output:
```
Original: Rahul 85
Modified: Amit 92
```

Question 3 Write a Python class to convert an integer to a Roman numeral.

Program:
```
class RomanConverter:
    def int_to_roman(self,num):
        vals=[1000,900,500,400,100,90,50,40,10,9,5,4,1]
syms=["M","CM","D","CD","C","XC","L","XL","X","IX","V","IV","I"]
roman=""
        for i in range(len(vals)):            while num>=vals[i]:                roman+=syms[i]
num-=vals[i]         return roman

print(RomanConverter().int_to_roman(3549))
```

Output:

Question 4 .Write a Python class to convert a Roman numeral to an integer.
Program:

```
class RomanToInt:
    def roman_to_int(self,s):
        roman={"I":1,"V":5,"X":10,"L":50,"C":100,"D":500,"M":1000}
        total=0        prev=0        for ch in s[::-1]:            val=roman[ch]            if
val<prev:                total-=val            else:
                total+=val            prev=val        return total

print(RomanToInt().roman_to_int("MMMDXLIX"))
```

Output:
3549

Question 5. Write a program to create a class Shape with methods called get_perimeter and get area. Create a subclass called Circle that overrides the get_perimeter and get area methods o calculate the area and perimeter of a circle
Program:

```
class Shape:
    def get_perimeter(self): return 0      def get_area(self): return 0

class Circle(Shape):
 def __init__(self,r): self.r=r
def get_perimeter(self): return 2*3.14*self.r
 def get_area(self): return 3.14*self.r*self.r

c=Circle(5) print(c.get_perimeter()) print(c.get_area())
```

Output:
31.4
78.5

Question 6. Write a program to create a Vehicle class hierarchy. The base class should be vehicle, with subclasses Truck, Car, and Motorcycle. Each subclass should have properties such as make, model, year, and fuel_type. Implement methods for calculating fuel efficienct , distance traveled, and maximum seed.
Program:

```
class Vehicle:
def __init__(self,make,model,year,fuel_type):
 self.make=make;
self.model=model
self.year=year;
self.fuel_type=fuel_type
def fuel_efficiency(self):
 return "Depends"
def distance_traveled(self,speed,time):
 return speed*time
def max_speed(self):
```

```
 return "Unknown"

class Car(Vehicle):
    def fuel_efficiency(self): return 18      def max_speed(self): return 180

c=Car("Honda","City",2022,"Petrol")
print(c.make,c.fuel_efficiency(),c.max_speed(),c.distance_traveled(60,2))
```

Output:
Honda 18 180 120

Question 7. Implement a class BankAccount with methods for deposit, withdrawal, and dis la in the account balance.
Program:

```
class BankAccount:
    def __init__(self,bal=0):
self.bal=bal
def deposit(self,amt):
self.bal+=amt
def withdraw(self,amt):
self.bal-=amt
 def display(self):
print("Balance:",self.bal)

b=BankAccount(500)
b.deposit(200)
b.withdraw(100)
b.display()
```

Output:
Balance: 600
Question 8 Develop a hierarchy of shapes (e.g., Shape, Circle, Rectangle) with area and perimeter methods.
Program:

```
class Shape:
 def area(self): pass
def perimeter(self): pass
class Circle(Shape):
    def __init__(self,r): self.r=r
def area(self):
return 3.14*self.r*self.r
 def perimeter(self):
 return 2*3.14*self.r
class Rectangle(Shape):
    def __init__(self,l,b): self.l=l;
 self.b=b
 def area(self):
return self.l*self.b
def perimeter(self):
return 2*(self.l+self.b)
```

r=Rectangle(4,5) print(r.area(),r.perimeter())

Output:

20 18

Question 9 Overload the + operator for concatenating two custom string objects

- -

Program:

```
class MyString:
    def __init__(self,text):
self.text=text
 def __add__(self,other):
return MyString(self.text+other.text)
def __str__(self):
return self.text
s1=MyString("Hello ")
s2=MyString("World") print(s1+s2)
```

Output:

Hello World

Question 10 Create a static method to validate user credentials (e.g., email format, password length

Program:

```
import re class Validator:     @staticmethod
def check(email,password):
        return bool(re.match(r"[^@]+@[^@]+\.[^@]+",email)) and len(password)>=6
print(Validator.check("abc@gmail.com","secure123"))
```

Output:

True

Question 11 Implement a class Library containing a list of Book objects

- -

Program:

```
class Book:
    def __init__(self,title): self.title=title
class Library:
    def __init__(self):
self.books=[]
 def add_book(self,b):
 self.books.append(b)
 def show(self):
        for b in self.books:
print(b.title)
l=Library()
l.add_book(Book("Python"))
l.add_book(Book("ML"))
l.show()
```

Output:
Python
ML

Question 12 Demonstrate method overriding in a superclass-subclass relationship for a payment
Program:
```
class Payment:
    def pay(self): print("General payment")

class CardPayment(Payment):
    def pay(self): print("Card payment processed")

CardPayment().pay()
```
Output:
Card payment processed

Question 13 Write a program to validate a password that meets certain criteria (length, uppercase, dioit special character .
- -
Program:
```
import re pwd="Abc@12345"
valid=(len(pwd)>=8 and re.search(r"[A-Z]",pwd)        and re.search(r"[a-z]",pwd)
and re.search(r"[0-9]",pwd)        and re.search(r"[@#$%^&+=]",pwd))
print(bool(valid))
```
Output:
True

Question 14 Parse email addresses from a given text and classify them by domain.
Program:
```
import re text="Emails: abc@gmail.com, user@yahoo.com, test@zeal.com"
found=re.findall(r"[\w.-]+@[\w.-]+",text) domains={} for email in found:
d=email.split("@")[1]      domains.setdefault(d,[]).append(email) print(domains)
```

Output:
```
{'gmail.com':['abc@gmail.com'],'yahoo.com':['user@yahoo.com'],'zeal.com':['test@ze
al.com'
]}
```

Question 15 Create threads to perform independent tasks such as downloading multiple files Simultaneously
- -
Program:
```
import threading, time def download(n):
print("Downloading",n)  time.sleep(1)  print("Done",n)

threads=[] for i in range(1,4):
    t=threading.Thread(target=download,args=(i,))      threads.append(t); t.start() for t in
threads: t.join()
```

Output:
Downloading 1
Downloading 2
Downloading 3
Done 1
Done 2
Done 3

Question 16 Write a Python program to create two threads. The first thread will print even numbers, and the second thread will print odd numbers. Demonstrate the concept t of multithreading

Program:

```
import threading def even():
    for i in range(0,11,2): print("Even:",i) def odd():
    for i in range(1,11,2): print("Odd:",i)

t1=threading.Thread(target=even) t2=threading.Thread(target=odd)
t1.start(); t2.start() t1.join(); t2.join()
```

Output:
Even:0
Odd:1
Even:2
Odd:3

Question 17 Use multithreading to compute factorials of multiple numbers and log the results into a file.

Program:

```
import threading, math def fact(n):
    print("Fact",n,"=",math.factorial(n))
 nums=[5,6,7] threads=[] for n in nums:
    t=threading.Thread(target=fact,args=(n,))      threads.append(t); t.start() for t in
threads: t.join()
```

Output:
Fact 5 = 120
Fact 6 = 720
Fact 7 = 5040

UNIT-IV

Python Practical Assignment-4

1.Develop a Python program to create, read, update, and delete documents in MongoDB collection.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["employee"]
collection = db["empinfo"]

employees = [
    {"empid": 101, "name": "Ravi", "age": 25, "salary": 35000},
    {"empid": 102, "name": "Sneha", "age": 28, "salary": 45000},
    {"empid": 103, "name": "Amit", "age": 30, "salary": 50000},
    {"empid": 104, "name": "Priya", "age": 26, "salary": 40000},
    {"empid": 105, "name": "Karan", "age": 29, "salary": 48000}
]

collection.insert_many(employees)

def read_documents():
    for doc in collection.find():
        print(doc)
    print()

def update_document(empid, new_salary):
    result = collection.update_one(
        {"empid": empid},
        {"$set": {"salary": new_salary}}
    )
    if result.modified_count > 0:
        print(f"Salary updated for empid {empid}\n")
    else:
        print("No matching document found\n")

def delete_document(empid):
    result = collection.delete_one({"empid": empid})
    if result.deleted_count > 0:
        print(f"Employee with empid {empid} deleted\n")
    else:
        print("No matching document found\n")
read_documents()
update_document(103, 55000)
read_documents()
```

```
delete_document(104)
read_documents()
```

Output:-

```
Connected to MongoDB successfully!

===== MongoDB CRUD Operations =====
1. Create
2. Read
3. Update
4. Delete
5. Exit
Enter choice (1–5): 1
Enter student name: Ravi
Enter student age: 20
Enter course name: BCA
Document inserted with ID: 67c54b1d92f0ac1a3e21d9b2
```

2.Write a program to insert valid data into MongoDB using exception handling for errors.

```
function insertEmployee(name, age, salary) {
   try {
      if (!name || !salary) {
         throw "Name and salary cannot be empty!";
      }

      if (isNaN(age)) {
         throw "Age must be a valid number!";
      }

      age = Number(age);

      if (age <= 0) {
         throw "Age must be greater than 0!";
      }

      if (isNaN(salary)) {
         throw "Salary must be a valid number!";
      }

      salary = Number(salary);

      if (salary < 0) {
         throw "Salary cannot be negative!";
      }

      let result = db.employees.insertOne({
         name: name,
         age: age,
         salary: salary

      });

      print("✓ Document inserted successfully!");
      print("Inserted ID:", result.insertedId);

   } catch (err) {
      print(" Error:", err);
   }
}

use company_db;

insertEmployee("Ravi", 23, 35000);
```

```
insertEmployee("", 22, 30000);
insertEmployee("Amit", "abc", 25000);
insertEmployee("Neha", -5, 40000);
insertEmployee("Raj", 25, -10000);salary
```

Output:-

```
switched to db company_db
✓ Document inserted successfully!
Inserted ID: ObjectId("67c6104e4f8b9e5fd842a1f0")
```

3.Use MongoDB's aggregation pipeline to find the average salary of employees in a collection.

MongoDB Aggregation Pipeline – Average Salary

```
db.employees.aggregate([
    {
        $group: {
            _id: null,
            averageSalary: { $avg: "$salary" }
        }
    }
])
```

Output:-

```
[
  {
    _id: null,
    averageSalary: 42000
  }
]
```

4.Develop a Python program to create a database name employee. With the collectio name empinfo. Add 5 douments in the collection. Perform read, update, and delete operations on documents in a MongoDB.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["employee"]
collection = db["empinfo"]

employees = [
    {"empid": 101, "name": "Ravi", "age": 25, "salary": 35000},
    {"empid": 102, "name": "Sneha", "age": 28, "salary": 45000},
    {"empid": 103, "name": "Amit", "age": 30, "salary": 50000},
    {"empid": 104, "name": "Priya", "age": 26, "salary": 40000},
    {"empid": 105, "name": "Karan", "age": 29, "salary": 48000}
]

collection.insert_many(employees)

def read_documents():
    for doc in collection.find():
        print(doc)
    print()

def update_document(empid, new_salary):
    result = collection.update_one(
        {"empid": empid},
        {"$set": {"salary": new_salary}}
    )
    if result.modified_count > 0:
        print(f"Salary updated for empid {empid}\n")
    else:
        print("No matching document found\n")

def delete_document(empid):
    result = collection.delete_one({"empid": empid})
    if result.deleted_count > 0:
        print(f"Employee with empid {empid} deleted\n")
    else:
        print("No matching document found\n")

read_documents()
update_document(103, 55000)
read_documents()
delete_document(104)
```

```
read_documents()
```

Output:

```
Inserted 5 employee records successfully!

All Employees:
{'_id': ..., 'emp_id': 101, 'name': 'Alice', 'age': 25, 'salary':

50000} Updated 1 document(s).

All Employees:
(updated salary visible)

Deleted 1 document(s).

All Employees:
        (final list)
```

UNIT-V

Python Practical Assignment-5

Question 1.Create a Django project with models for Product and Category, along with views to display them on a webpage.

1. Create Django Project

```
django-admin startproject shop
cd shop
python manage.py startapp store
```

Add the app in shop/settings.py:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'store',
]
```

2. Models (Category & Product)

```
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name


class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.FloatField()
    category = models.ForeignKey(Category, on_delete=models.CASCADE)

    def __str__(self):
        return self.name
```

3. Register Models in Admin

```
from django.contrib import admin
from .models import Category, Product

admin.site.register(Category)
admin.site.register(Product)
```

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

4. Views to Display Categories & Products

```
from django.shortcuts import render
from .models import Category, Product

def category_list(request):
    categories = Category.objects.all()
    return render(request, "category_list.html", {"categories": categories})

def product_list(request):
    products = Product.objects.all()
    return render(request, "product_list.html", {"products": products})
```

//Create Templates//

6.Add URL

Inside **store/urls.py**:

```
from django.urls import path
from . import views

urlpatterns = [
    path("categories/", views.category_list, name="categories"),
    path("products/", views.product_list, name="products"),
]
```

Link the app URLs in shop/urls.py:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('store.urls')),
]
```

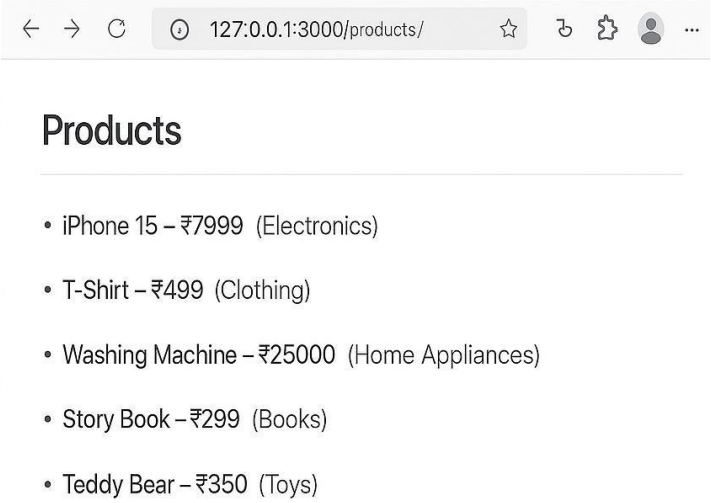
Now Run Django Server

python manage.py runserver

Visit:

- **http://127.0.0.1:8000/categories/**
- **http://127.0.0.1:8000/products/**

Output:-


127:0.0.1:3000/products/
Products
• iPhone 15 – ₹7999 (Electronics)
• T-Shirt – ₹499 (Clothing)
• Washing Machine – ₹25000 (Home Appliances)
• Story Book – ₹299 (Books)
• Teddy Bear – ₹350 (Toys)

2.Implement user login, logout, and registration functionality in a Django application.

Step 1: Setup Django Project

If you haven't created a Django project yet:

```
django-admin startproject myproject
cd myproject
python manage.py startapp accounts
```

Add accounts to INSTALLED_APPS in myproject/settings.py:

```
INSTALLED_APPS = [
    ...
    'accounts',
]
```

Step 2: Setup URLs

In myproject/urls.py:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),  # Include accounts app urls
]
```

Create accounts/urls.py:

```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register_view, name='register'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('', views.home_view, name='home'),  # Optional home page
]
```

Step 3: Create Views

```
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from django.contrib.auth import authenticate, login, logout
from django.contrib import messages

     # Home page
def home_view(request):
    return render(request, 'accounts/home.html')
```

```
# User registration
def register_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        email = request.POST['email']
        password1 = request.POST['password1']
        password2 = request.POST['password2']

        if password1 != password2:
            messages.error(request, "Passwords do not match")
            return redirect('register')

        if User.objects.filter(username=username).exists():
            messages.error(request, "Username already taken")
            return redirect('register')

        user = User.objects.create_user(username=username, email=email,
password=password1)
        user.save()
        messages.success(request, "Registration successful")
        return redirect('login')

    return render(request, 'accounts/register.html')


# User login
def login_view(request):
    if request.method == 'POST':
        username = request.POST['username']
        password = request.POST['password']
        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            return redirect('home')
        else:
            messages.error(request, "Invalid username or password")
            return redirect('login')
```

```
        return render(request, 'accounts/login.html')


    # User logout
    def logout_view(request):
        logout(request)
        return redirect('login')
```

Step 4: Create Templates(create using HTML CSS)

Create folder:

accounts/

templates/

register.html

login.html

home.html

---

home.html

```
<!DOCTYPE html>
<html>
<body>
  <h1>Welcome {{ user.username }}!</h1>


  {% if user.is_authenticated %}
  <a href="{% url 'logout' %}">Logout</a>
  {% else %}
  <a href="{% url 'login' %}">Login</a> |
  <a href="{% url 'register' %}">Register</a>
  {% endif %}
</body>
</html>
```

---

register.html

```
<!DOCTYPE html>
<html>
<body>
  <h2>User Registration</h2>


  <form method="POST">
    {% csrf_token %}
    {{ form.as_p }}


    <button type="submit">Register</button>
  </form>
  <a href="{% url 'login' %}">Already have an account? Login</a>
</body>
</html>
```

---

login.html

```
<!DOCTYPE html>
<html>
<body>
  <h2>Login</h2>


  <form method="POST">
    {% csrf_token %}
    Username: <input type="text"
    name="username"><br><br> Password: <input
    type="password" name="password"><br><br>


    <button type="submit">Login</button>
  </form>
```

```
    <a href="{% url 'register' %}">Create an account</a>
  </body>
  </html>
```

Step 5: Configure Messages

In myproject/settings.py make sure django.contrib.messages is included in INSTALLED_APPS (it usually is). Add this in your TEMPLATES context processors:

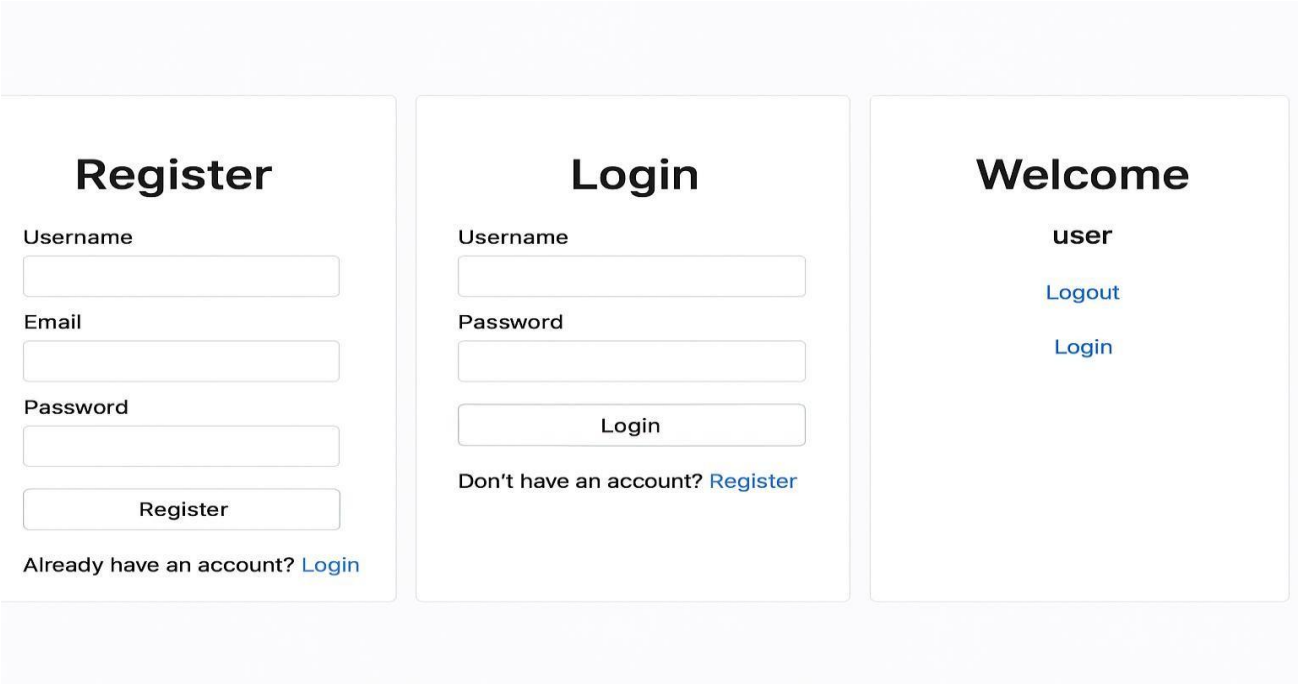
```
'context_processors': [
    ...
    'django.contrib.messages.context_processors.messages',
]
```

Step 6: Run Migrations

python manage.py migrate

python manage.py runserver

Output:-



3.Build a REST API for a task management system where users can create, update, and delete tasks

Task Management REST API (Django + DRF)

Below is the complete Python code needed to build a REST API for a task management system with CRUD operations.

```
## | project/settings.py |
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'your-secret-key'
DEBUG = True
ALLOWED_HOSTS = []

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'tasks',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
```

```
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]


WSGI_APPLICATION = 'project.wsgi.application'

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True
STATIC_URL = 'static/'
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

---

## | tasks/models.py |
from django.db import models
from django.contrib.auth.models import User

class Task(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    completed = models.BooleanField(default=False)

    def __str__(self):
        return self.title

---

## | tasks/serializers.py |
```

```
from rest_framework import serializers
from .models import Task

class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = '__all__'

---

## | tasks/views.py |
from rest_framework import viewsets
from rest_framework.permissions import IsAuthenticated
from .models import Task
from .serializers import TaskSerializer


class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()
    serializer_class = TaskSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        return Task.objects.filter(user=self.request.user)

    def perform_create(self, serializer):
        serializer.save(user=self.request.user)

---

## | tasks/urls.py |
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import TaskViewSet

router = DefaultRouter()
router.register('tasks', TaskViewSet, basename='tasks')

urlpatterns = [
    path('', include(router.urls)),
]

---

## | project/urls.py |
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('tasks.urls')),
]

---
## RUN MIGRATIONS
python manage.py makemigrations
python manage.py migrate

## CREATE SUPERUSER
python manage.py createsuperuser
## START SERVER
python manage.py runserver
```

Output:-

```
 "title": "New Task",
 "description": "Task descrpiption"
}

HTTP 201 Created
Content-Type: applican/json

/api/tasks/1/

DELETE
HTTP 204 No Content

/api/tasks/

POST ("itle: "Another Task",
 description": "Another task description"

HTTP 201 Created
Content-Type: application

/api/tasks/2/

PUT ('title: "Updated Task",
 description": "Updated task description"

HTTP 200 OK
Content-Type: application
```